13.1 Weighted graph 與 shortest path

有限有向加權圖（directed weighted graph）$G = (V, E, w)$ 的路徑成本是邊權重之和。允許重複頂點的 walk 若能經過負權環再抵達 v，就可任意多繞該環，使到 v 的成本沒有下界。若 v 不可由該環抵達，其最短距離不因此受影響。本章 Dijkstra 的通用正確性前提是從 source 可達的所有邊都滿足 $w(e) \geq 0$。

$d[v]$ 是目前已知的上界，$\pi[v]$ 是 predecessor。初始化為 $d[s] = 0$，其他 $d[v] = \infty$。

若圖是有向無環圖，可用 topological relaxation；若有 negative weights 但沒有 negative cycle，則用 Bellman-Ford。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.2 Dijkstra invariant 與正確性

每輪選 $d[u]$ 最小且尚未 finalized 的頂點 u。不變量是已 finalize 頂點的 d 已是真正的 shortest distance；有限距離頂點的 π 記錄一條 source path；不可達頂點維持無窮大與空 predecessor。

鬆弛（relax）edge $u \to v$：

$$d[v] = \min(d[v], d[u] + w(u, v)).$$

Dijkstra 的成本取決於 priority queue：array 掃描最小值 n 次為 O(n^2)，另掃 m 條弧；binary heap 版本為 $O((n+m)\log n)$。Fibonacci heap 版初始化與 insert 共 O(n)，至多 n 次 extract-min 各攤銷 O(log n)，至多 m 次成功 decrease-key 各攤銷 O(1)，加上逐弧鬆弛 O(m)，所以總成本 $O(m + n\log n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra 正確性：第一個未確定的點

以反證法說明正確性。若本輪最小 key 的 u 滿足 $\mathrm{dist}(s,u) < d[u]$，取一條最短 $s \to u$ 路徑上首個尚未 finalized 的頂點 x，其 predecessor y 已 finalized。由先前的鬆弛，

$$d[x] \le \mathrm{dist}(s,y) + w(y,x) = \mathrm{dist}(s,x).$$

非負邊權使

$$\mathrm{dist}(s,x) \le \mathrm{dist}(s,u) < d[u],$$

這與 u 是最小 key 矛盾。若 $x = s$，起點已有 $d[s] = 0$，也不可能出現這種錯誤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra 的逐輪數值與 predecessor

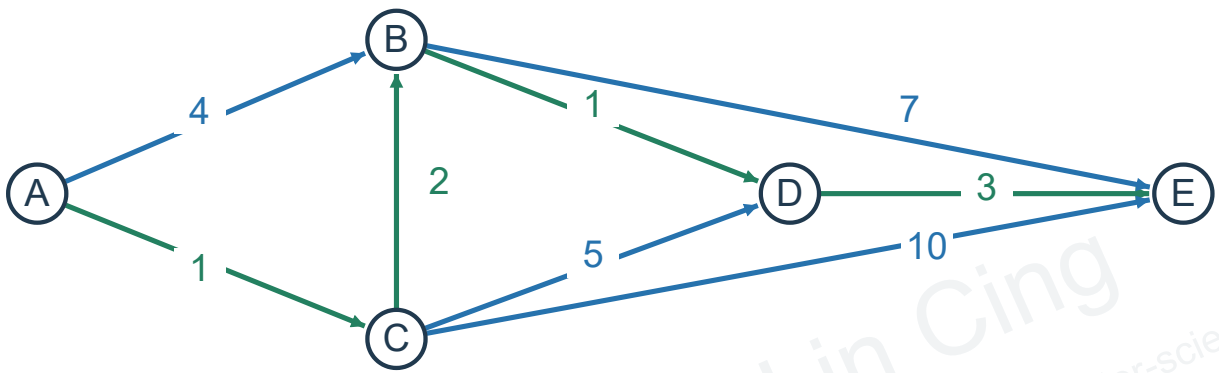
固定弧 A→B:4、A→C:1、C→B:2、B→D:1、C→D:5、D→E:3、C→E:10、B→E:7。Source 為 A，每輪取尚未確定的最小 d，鬆弛其所有出弧。

確定頂點後	d[A]	d[B]	d[C]	d[D]	d[E]	此輪改善的 predecessor
初始化	0	∞	∞	∞	∞	全空
A	0	4	1	∞	∞	B←A、C←A
C	0	3	1	6	11	B←C、D←C、E←C
B	0	3	1	4	10	D←B、E←B
D	0	3	1	4	7	E←D
E	0	3	1	4	7	無

E 的 predecessor 鏈反向為 E←D←B←C←A，還原 A–C–B–D–E，成本 1+2+1+3=7。Heap 版本只在嚴格改善時更新；用多份 key 的 lazy heap 時，取出舊 key 必須跳過，不能重複將它視為新的最小距離。

邊權與最終距離是不同的數量

Dijkstra 同一有向圖：弧權重、最終距離與 predecessor tree



確定次序：A、C、B、D、E

d(A,B,C,D,E) = (0,3,1,4,7)

到 E 的 path：

A → C → B → D → E

成本 1+2+1+3 = 7

箭頭旁數字為原弧權重；綠色是最終 predecessor tree。距離由右側 d 表另列。

圖中的 C→D 權重為 5，但 d[D]=4，因為最短路徑改走 C→B→D，成本 2+1=3，加上 A→C 的 1 才為 4。不能把某條弧的權重直接當成抵達終點的總距離。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

最短距離的證明憑證與負邊邊界

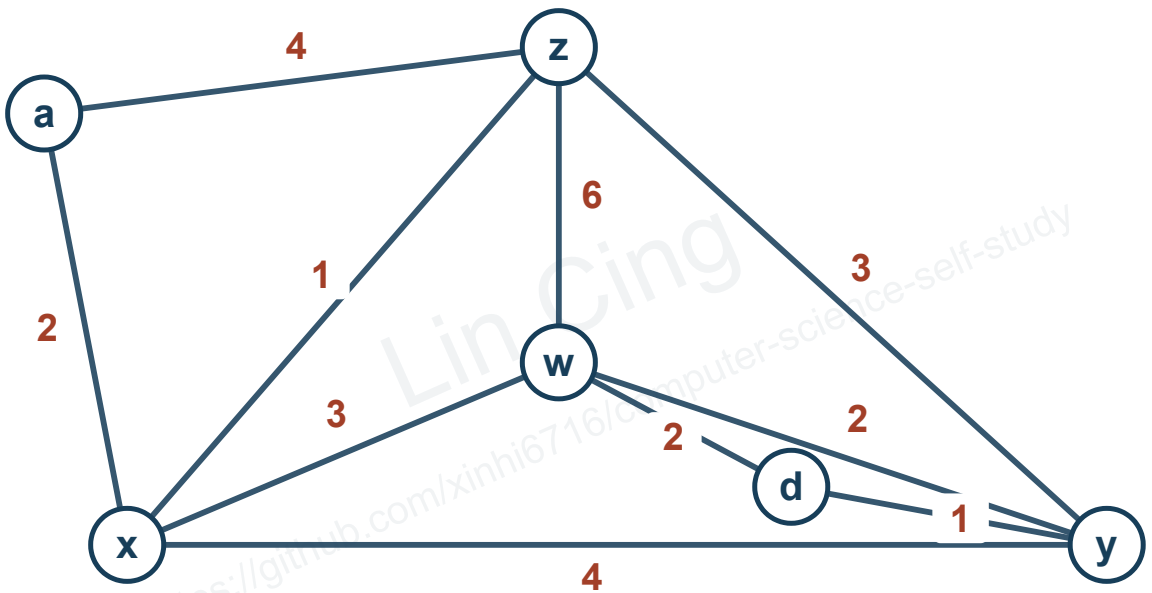
若 d[s]=0，對每條可達弧 u→v 有 $d[v] \le d[u] + w(u,v)$，而每個有限 d[v] 又有一條 predecessor path 達到此成本，便可認證最短性。沿任意 s→v 路徑逐邊套不等式，得 d[v] 不大於該路徑成本；predecessor path 又給達到此界的實際路徑，兩者相等。

不可達頂點需另以圖搜尋證明不可達，不能只因某次 heap 沒更新就猜測。若 extract-min 得 ∞，剩餘點都不可達，可停止。已確定點不再變更 predecessor，可避免零權邊相互改父造成不必要的迴圈。

負權邊的反例 s→a:2、s→b:1、a→b:−2 沒有負環；先固定 b=1 會錯過 0。若要接受一般負權邊，Bellman–Ford 在第 k 輪用上一輪的所有距離鬆弛，得到「至多 k 條邊的最短 walk 成本」。沒有可達負環時，最短 walk 可去掉非負 cycle 成為至多 n−1 邊的 path，因此 n−1 輪足夠；第 n 輪仍改善，才指出可達負環。其受影響的下游點需再搜尋傳播，不是所有頂點一律無最短距離。

同一張圖：最短路徑與最小生成樹

以下圖為有限無向圖，邊上的數字是非負權重；以 a 為 Dijkstra 的起點。



每輪距離相同時依頂點字母序選定；只有距離**嚴格變小**才更新 predecessor，等長時保留原值。Kruskal 對等權邊依端點字母序處理。

Dijkstra：逐輪比較與更新

表中 a, x, z, w, y, d 的距離依此順序列出；未成功的嘗試仍要比較，而不是跳過相鄰邊。

確定點	對尚未確定點的嘗試	更新後距離 (a, x, z, w, y, d)
a	x:2，z:4	(0,2,4,∞,∞,∞)
x	z:3<4，w:5，y:6	(0,2,3,5,6,∞)
z	w:9>5，y:6=6	(0,2,3,5,6,∞)
w	y:7>6，d:7	(0,2,3,5,6,7)
y	d:7=7	(0,2,3,5,6,7)
d	無尚未確定鄰居	(0,2,3,5,6,7)

嚴格改善產生的前驅為 $x \leftarrow a, z \leftarrow x, w \leftarrow x, y \leftarrow x, d \leftarrow w$；確定次序為 a, x, z, w, y, d。例如 $a \to x \to z \to y$ 與 $a \to x \to y$ 同長 6，但等長不更新，因此 y 保留 x 作前驅。

Kruskal：依權重合併不同分量

在等權邊中，按兩端點排序後的字母序處理；有五條選邊後即停止。

考察邊	權重	決定及理由
dy	1	選；合併 d、y
xz	1	選；合併 x、z
ax	2	選；a 加入 xz 分量
dw	2	選；w 加入 dy 分量
wy	2	略過；w、y 已屬同一分量
wx	3	選；合併兩個分量，完成生成樹

所選 $\{dy, xz, ax, dw, wx\}$ 的總權重是 $1+1+2+2+3=9$。每條選邊跨越當時不同的分量；在所有跨該分量切割的邊中，已排序次序保證其權重最小，故可由 cut property 證明安全。

比較目標與驗證結果

上面的 Dijkstra 前驅樹為 $\{ax, xz, xw, xy, wd\}$，其邊權和是 $2+1+3+4+2=12$；從 a 到各點的樹上距離依序為 $0, 2, 3, 5, 6, 7$。Kruskal 樹的邊權和較小，為 9，但其 $a \to x \to w \to d \to y$ 長度為 $2+3+2+1=8$，大於最短距離 6。

兩者都是生成樹：最短路徑樹要求各頂點的根到點距離最小；最小生成樹要求所選邊的**總權重**最小。平手可能產生其他合法樹，不能把這個特定的 tie-breaking 結果視為唯一答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.3 Kruskal 與 cut property

Kruskal 依 weight 排序 edges，若加入後不形成 cycle 就接受，否則丟棄。minimum spanning tree 的 cut property 指出，對非空 $S \subset V$，任何最小 crossing edge 都可被某棵 MST 使用。

交換證明：若 MST T 不含 cut 的最小 crossing edge e，在 T 中連接 e 兩端的唯一路徑上，必有 crossing edge f。加入 e 後，該路徑與 e 形成唯一 cycle；刪掉 f 才能保證換邊後仍是樹。由 e 的最小性，$w(f) \geq w(e)$。把 f 換成 e 後仍連通、仍有 $n-1$ edges，所以仍是 spanning tree，總重不增。若 T 原本是 MST，換後也不低於它，因此換後仍是 MST。

例：cycle $A-B(2), B-C(1), C-A(4)$ 加 link C-D(3)。Kruskal 取 BC,AB,CD，總 6，避開權重 4 cycle edge。

Edge weights 相等時 MST 可能不唯一，但最小總重不變。

13.4 Prim 與 invariant

Prim 從任意 root 開始，每次加入連接 current tree S 與 $V \setminus S$ 的最小 crossing edge。Invariant：已加入 edges 是某 MST 的 subset。

其正確性使用尊重已選邊集合的 cut property。Kruskal 每次取某一 forest component 與外界的最輕邊，同樣保持可擴成某棵 MST 的不變量。Cycle property 指出，若 e 是某 cycle 中唯一最重邊，則 e 不屬於任何 MST；否則在去掉 e 所得 cut 上，可選該 cycle 的較輕 crossing edge 換入而降低成本。若最重邊有 ties，仍存在不含指定最重邊的 MST，但不能說所有 MST 都排除它。Prim 維護一個 growing tree；Kruskal 維護 forest。

Disconnected graph 沒有 spanning tree；需先判斷 connectivity，或輸出 spanning forest。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

13.5 Max-flow network 的模型

有限 directed network $G = (V, E)$ 有不同的 source s、sink t，每條弧容量 $c(e) \geq 0$。Flow $f(e)$ 是各原始弧上非負的量，滿足：

$$0 \leq f(e) \leq c(e),$$

每個 $v \neq s, t$ 的 in-flow 等於 out-flow。Total flow 是 source 的 out-flow 減去 in-flow。

若 edge 是 undirected，需決定模型為兩條 directed arcs 或 single shared capacity；兩者不能混用。

Anti-parallel edges 可在 residual construction 中用不同 residual amounts 表示，不會因符號合併而消掉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.6 Residual graph 與 augmenting path

對每條有獨立 ID 的原始弧 e=(u,v)，建立正向、反向兩個 residual records，其容量分別是

$$c_f(e^+) = c(e) - f(e), \qquad c_f(e^-) = f(e).$$

e^+ 方向為 u→v，e^- 方向為 v→u，後者表示可取消多少原流量。若 s 到 t 的 augmenting path 有 bottleneck $\Delta > 0$，沿 path 的正向 record 增加對應原流，反向 record 減少對應原流，仍保持流量守恆與容量限制。同一方向可能有不同來源的 records，不能只以端點對覆蓋它們。

若新增不必要的 opposite arcs，需用 net flow 轉換；程式通常存 residual capacities 並同步更新 pair。

Residual path 不等於原 graph 的 directed path；它描述還可以如何調整流量。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

13.7 Max-flow min-cut

Cut (S,T) 是 V 的分割，滿足 $s \in S, t \in T$，capacity 只加原始 $S \to T$ 弧的容量。把 S 中所有點的流量守恆式相加，內部弧一正一負抵銷，只剩

$$|f| = \sum_{u \in S, v \in T} f(u,v) - \sum_{u \in T, v \in S} f(u,v) \le \sum_{u \in S, v \in T} c(u,v) = c(S,T).$$

若不存在 augmenting path，令 S 為 residual graph 從 s 可達的點，則 t∉S。對每條原始 S→T 弧，正向殘餘容量必為 0，故已飽和；對每條原始 T→S 弧，其反向殘餘弧朝 S→T，因此流量必為 0。代入上式便有 |f|=c(S,T)，同時認證最大流與最小割。原始 T→S 弧的「正向」殘餘容量仍可大於 0，不能把所有跨割反向弧一概說成零。

整數 capacities 且每次從零流沿增廣路徑增加整數 bottleneck，流值每次至少增加 1，又有有限 cut 上界，因此會終止並產生整數最大流。任意實數 capacities 的任意路徑選法不保證有限步終止；Edmonds–Karp 選最少邊數的增廣路徑可保證多項式次數。Min-cut 可有多個，但 max-flow 的 value 唯一。Cut capacity 的方向依 convention 而定，作答前必須固定。

13.8 Bipartite matching

二分圖 $G = (L \cup R, E)$ 的 matching 是一組邊，任兩條不共用端點。若 L 的每個點都被匹配，稱為 saturating L；perfect matching 則要求 L、R 所有頂點都被匹配。因此只飽和 L 而 |L|<|R| 時並不 perfect。對 $S \subseteq L$，令 $N(S)$ 為 R 中與 S 至少一點相鄰的全部鄰點。

Hall theorem（霍爾定理）指出，此圖存在一個匹配飽和全部 L，若且唯若

$$\forall S \subseteq L, \quad |N(S)| \geq |S|.$$

必要性是：S 的 $|S|$ 個 endpoints 必須配到相異 neighbors。充分性取 maximum matching M。若有未匹配左點，從所有這類點沿非匹配 $L \to R$、匹配 $R \to L$ 的 edges 搜尋；若到未匹配右點，便有 augmenting path，與最大性矛盾。否則令可達左、右集合為 S, T，所有 $N(S)$ 都在 T，T 的各點皆匹配到 S，且 S 另含至少一個未匹配起點，故

$$|N(S)| = |T| < |S|,$$

違反 Hall 條件。

Hall condition 是全域條件，必須檢查每個 subset；實作使用 augmenting-path algorithm，而不是逐 subset 枚舉。

13.9 Matching 與 flow 轉換

Unweighted bipartite max matching 可轉成 unit-capacity network：source 連到每個 L，原有的 $L \to R$ edge 保留，每個 R 連到 sink，全部 capacities 為 1。Integral max-flow 給出 matching；matching size 等於 flow value。

Minimum vertex cover 在 bipartite graph 與 maximum matching 同 size（Kőnig）。加權指派需將成本明確納入目標，例如使用最小成本流或相應指派演算法；只有容量限制與最大匹配數不足以決定最低成本。

Integrality 依 capacities 為整數；實數 capacitated 問題不保證整數 flow，但 bipartite reduction 的 arcs 是 unit。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.10 常見錯誤與反例

- Dijkstra 的 negative edge 反例：$s \to a = 2, s \to b = 1, a \to b = -2$；$b$ 先 finalized，之後由 a 得到更小的 0，卻不能再更新。
- Kruskal 不檢查 cycle 可能在取滿 n−1 條時仍含 cycle 且不連通，或加入更多冗邊；邊數不固定為 n。
- 計算 max-flow cut capacity 時只加 forward capacities。
- Hall condition 不能只查 $\forall x, |N(x)| \geq 1$。
- Perfect matching 要求兩側全部被匹配；最大 matching 仍可能留下未匹配點。

通常錯誤來自忽略 model assumptions，而不是 greedy choice 本身。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

安全邊必須尊重已選 forest

假設 F 已包含在某棵 MST 中。若 cut (S,V−S) 沒有任何 F 的邊跨越它，稱此 cut 尊重 F。令 e 是該 cut 的最輕跨邊。若某棵包含 F 的 MST T 不含 e，將 e 加入 T 的唯一 cycle 必有另一條跨邊 f；因 cut 尊重 F，f∉F。用 e 換 f 不增加成本，也沒有刪掉 F 的邊，所以 F∪{e} 仍可擴成 MST。

這個「保留 F」的版本才直接證明 greedy 演算法。Kruskal 考慮一條連接不同分量的下一最輕邊時，取其中一個分量為 S：較輕且已掃過的跨邊不可能被拒絕，否則兩端當時已同分量、現在也不會跨此 cut。因此 e 對此 cut 最輕。Prim 則直接選目前已加入點集 S 的最輕跨邊。初始 F 空可擴成 MST，每次維持，最後連通且 n−1 邊，便是 MST。

MST 權重可以為負或零，交換證明只比較權重大小，沒有使用非負性。不要把 Dijkstra 的前提錯套到 MST。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kruskal 與 Prim 的同一圖追蹤

無向邊 AB:1、AC:4、BC:2、BD:5、CD:3。Kruskal 的完整掃描如下；接受到三邊即可提前停止，表中仍列其餘兩邊供檢查 cycle。

邊	決定	接受後分量
AB:1	接受	{A,B}、{C}、{D}
BC:2	接受	{A,B,C}、{D}
CD:3	接受	{A,B,C,D}
AC:4	拒絕，A–B–C–A 成環	不變
BD:5	拒絕，B–C–D–B 成環	不變

https://github.com/xinhi6716/

Prim 的跨割候選與兩法成本

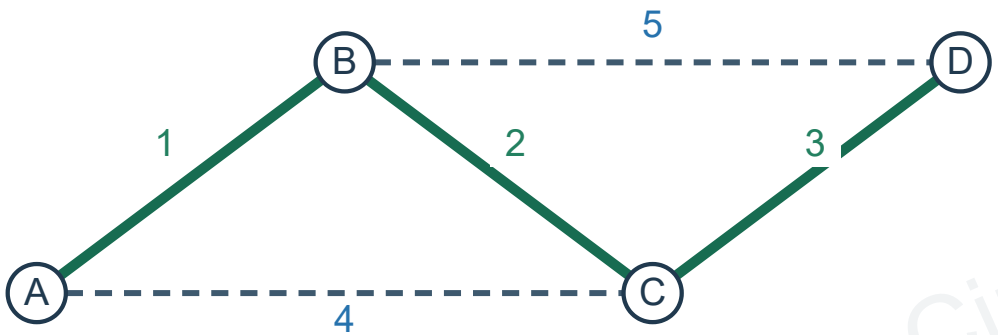
Prim 從 A 開始：S={A} 的候選 AB:1、AC:4，選 AB；S={A,B} 的候選 BC:2、AC:4、BD:5，選 BC；S={A,B,C} 的候選 CD:3、BD:5，選 CD。兩法都得總重 6。最後一個 cut 有兩條跨邊，不能漏掉 BD。

排序加 DSU 的 Kruskal 時間為 $O(n + m \log m + m\alpha(n))$：先初始化 n 個點，再排序 m 條邊並處理 DSU operations。含孤立點或不連通輸入時，不能把 O(n) 初始化漏掉。Prim 的 binary heap 鄰接表版本 $O((n + m) \log n)$，array 版本 $O(n^2 + m)$。Disconnected 圖應輸出每個分量的 minimum spanning tree，不能補不存在的邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

圖上核對所選邊與 cut 候選

同一圖的 MST：只依 cut 選邊，不依單一起點距離



接受 AB → BC → CD

總重 1+2+3 = 6

最後 cut：{A,B,C} | {D}

候選 CD:3、BD:5，選 CD。

粗實線：選中邊；虛線：其他原圖邊。BD 仍是最後 cut 的候選。

CD 是最後 cut 的唯一最輕邊，所以每棵 MST 必包含它。理由是若 T 不含 CD，其跨 cut 路徑必改用較重的 BD；交換後嚴格改善。這裡的「唯一最輕」不能省略成任意 tied 最輕邊都出現在每棵 MST。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Residual edge ID 與反向更新

對每條原始弧 e=(u,v,c)，獨立建立一對 residual records：正向初值 c，反向初值 0，兩者互存 rev index。增廣 Δ 時，所走 record 的 residual 減 Δ，它的配對 record 加 Δ。反向 record 表示取消先前那條原弧的部分流，不是創造負容量。

若原圖同時有 u→v 與 v→u，需有兩對、共四個 records；其中某對的反向 record 不能與另一原弧的正向 record 當成同一條弧。如果只為搜尋而合併同向殘餘容量，總量為 $c(u,v) - f(u,v) + f(v,u)$，但還原原弧流量時仍需知道各自的來源。

原模型中每條弧流量非負。若改用 skew-symmetric net flow，則須同步改容量限制與 value 公式；不能在一行用非負原弧流，下一行又把反向流量自動視作負值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一次取消舊流量的完整增廣

取全部容量為 1 的原弧 s→a、s→b、a→b、a→t、b→t。先沿 s–a–b–t 送 1，得到流量向量

$$(f_{sa}, f_{sb}, f_{ab}, f_{at}, f_{bt}) = (1, 0, 1, 0, 1).$$

此時 s→a 與 b→t 已滿，但仍有殘餘路徑 s→b→a→t；中間 b→a 是 a→b 的反向 record。Bottleneck 為 1，更新後

$$(1, 1, 0, 1, 1),$$

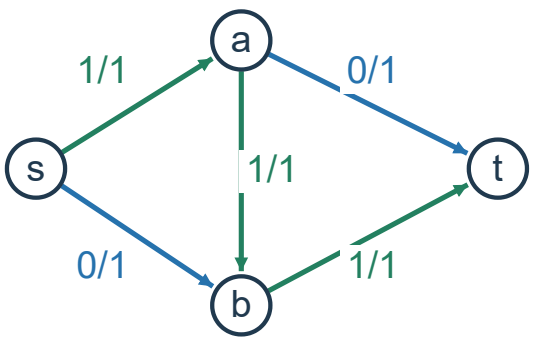
流值從 1 變 2，a→b 的流被取消，而不是把原圖不存在的 b→a 當作新運輸弧。割 {s} 的容量 2，認證此流已最大。若只沿原圖未飽和弧搜尋，就會錯把第一個流值 1 當成最大。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

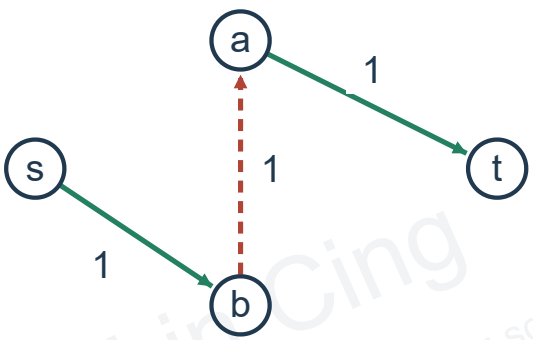
反向 residual 弧如何改寫原流

殘餘增廣的前態、操作與後態：中間弧取消原流量

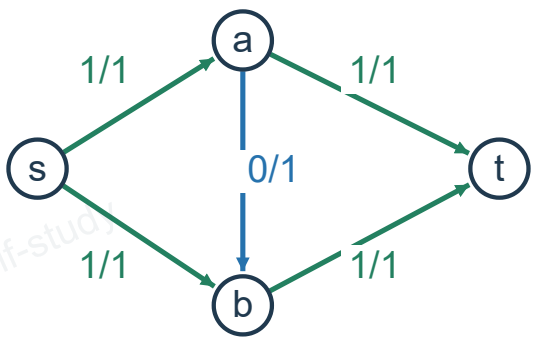
前態：原弧 f/c，|f|=1



操作：只畫增廣路徑 Δ=1



後態：原弧 f/c，|f|=2



中間圖數字為 residual capacity；紅色 b→a 取消 a→b 的流。後態 a→b 回到 0/1。

左、右兩圖是原始弧的 f/c；中間圖只畫本次走過的三條 residual records，數字是殘餘容量，不是原始 f/c。左圖 a→b 的 1/1 產生中圖 b→a 的容量 1，更新後對應右圖 a→b 的 0/1。三張圖的 a、b 與其他原弧保持同一位置。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Edmonds–Karp 的有限步保證

反覆用 BFS 在正 residual records 中找最少邊數 s→t 路徑；不存在就輸出當前流與可達割。找到時先沿 predecessor records 計 bottleneck，再沿同一路更新配對容量。BFS 邊長一律為 1，與 capacity 大小無關。

令 d(v) 為本輪 residual BFS 距離。增廣後新增的 record 只能是原最短路徑邊的反向；對這種新邊 u→v，舊距離有 d(v)=d(u)−1。原已存在的 record 則有 d(v)≤d(u)+1。所以沿任何新 residual s→x 路徑逐邊累加，舊 d(x) 不超過新路徑長度，證明距離不下降。原不可達點若變可達，取其新路徑上第一個舊不可達點也得到同樣矛盾。

某 record u→v 被瓶頸飽和時有 d(v)=d(u)+1；它若要再被飽和，期間必先沿反向 v→u 送流。到那時由距離不下降，

$$d_{\text{later}}(u) = d_{\text{later}}(v) + 1 \geq d_{\text{old}}(v) + 1 = d_{\text{old}}(u) + 2.$$

有限可達距離至多 n−1，因此每個 record 只會成為瓶頸 O(n) 次；共 O(m) records、每次增廣至少飽和一個，得 O(nm) 次增廣。忽略無關孤立點後一次 BFS 為 O(m)，核心時間 $O(nm^2)$，另加初始讀圖 $O(n+m)$。此證明不依容量為整數，但假設精確算術；浮點比較與容許誤差需另定規格。

Maximum、maximal 與增廣路徑

Maximum matching 是邊數全域最大；maximal matching 只要求不能再直接加一邊。例 L={a,b}、R={1,2}、邊 a1、a2、b1：M={a1} 已 maximal，卻非 maximum。交錯路徑 b–1–a–2 依序使用非匹配、匹配、非匹配邊，對其邊做對稱差，得 {b1,a2}，大小增加 1。

Berge 判準。 M 為 maximum，若且唯若沒有 augmenting path。若有增廣路徑，對稱差直接改善。反之取較大的 matching M′；M△M′ 每點 degree 至多 2，分解為交錯 paths 與偶 cycles。Cycles 及偶長 paths 兩種 matching 邊數相同；總體 M′ 較大，必有一條 path 的 M′ 邊比 M 多 1，其兩端都未被 M 匹配，正是 augmenting path。

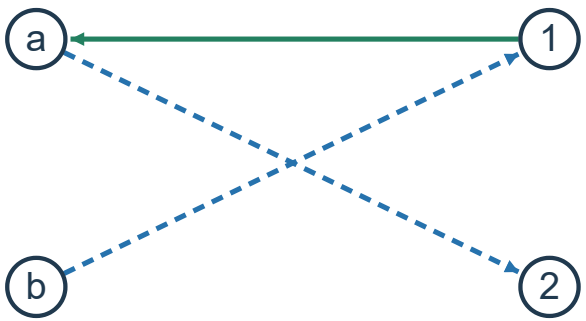
因此「找不到可直接加的邊」不夠，必須搜尋可取消再重配的整條交錯路徑。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

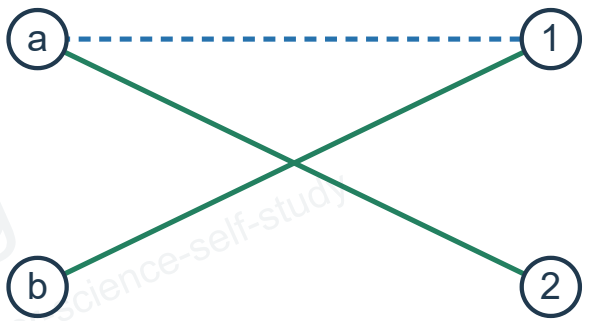
同一條交錯路徑的匹配翻轉

交錯路徑 b–1–a–2：翻轉邊的歸屬，匹配數由 1 變 2

前態 M={a1}，沿交錯方向搜尋



後態 M′={b1,a2}



綠實線為 matching；藍虛線為非 matching 邊。邊在中間相交處沒有頂點。

前態已配對的 a1 要沿反方向 1→a 走，因為要取消其匹配；另外兩邊沿左到右走。翻轉後每個內部點失去一條匹配邊又獲得一條，兩個未匹配端點各獲得一條，所以仍為 matching 且恰多 1 邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kőnig 定理的可建構證明

令 M 為二分圖最大匹配，從所有未匹配左點沿非匹配 L→R、匹配 R→L 邊搜尋，得到可達集合 Z_L、Z_R。由最大性，所有 Z_R 中的點都有匹配。取

$$C = (L \setminus Z_L) \cup Z_R.$$

這是 vertex cover：若邊左端不在 Z_L，已被第一部分覆蓋；若左端在 Z_L，非匹配邊會使右端可達，匹配邊的右端則是到達該左點的前一步（起始未匹配左點沒有此種邊），右端必在 Z_R。

每個 Z_R 的匹配左端在 Z_L；每個不在 Z_L 的左點也必已匹配，否則是搜尋起點。這兩部分恰把 M 的邊各計一次，因此 |C|=|M|。任何 vertex cover 至少要為每條彼此不共端的 matching edge 選一個端點，故 |cover| ≥|M|，證得 minimum cover = maximum matching。

前例最大匹配 {b1,a2} 飽和 L，沒有未匹配左點，所以 Z_L=Z_R=∅，C=L，大小 2。若只有 a1、b1，起點 b 可達 1、a，得到 Z_L={a,b}、Z_R={1}，minimum cover={1}。

Hall deficiency 與失敗憑證

對任意 $S \subseteq L$，最多 |N(S)| 個 S 中的點可匹配，所以任何匹配至少留下 |S|−|N(S)| 個未匹配左點。令 ν 為最大匹配大小，必有

$$|L| - \nu \geq \max_{S \subseteq L} (|S| - |N(S)|).$$

反向取 maximum M 的交錯可達集合 S=Z_L、T=Z_R；每個 T 都匹配到 S，S 除這些 matched mates 外，恰另有所有 k=|L|−ν 個未匹配起點。因此 |S|−|T|=k。又 N(S)=T，故這一組 S 達到上界，得到等號。若 k=0，取空集也達到 0。

例如左點 a、b、c、d；a、b、c 都只鄰接 1、2，d 只鄰接 3。S={a,b,c} 的 deficiency 為 1；匹配 {a1,b2,d3} 達到大小 3，所以恰留 1 個左點。這同時交付可行匹配與不可再大的證明，不是只列 Hall 失敗的口號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Regular 二分圖必有 perfect matching

若有限非空二分圖每點 degree 恰為 r≥1，對任意 S⊆L，從 S 發出的 r|S| 條邊全部抵達 N(S)；每個右鄰點總 degree 只有 r，因此

$$r|S| \le r|N(S)| \quad \Longrightarrow \quad |S| \le |N(S)|.$$

Hall 保證飽和 L 的匹配。由總邊數 r|L|=r|R|，又有 |L|=|R|，所以此匹配同時飽和 R，是 perfect matching。r=0 不能消去，含孤立點的零正則圖未必有 perfect matching。

刪除此 perfect matching 後，每點 degree 恰減 1。重複直到 degree 0，便把原邊集分解成 r 個彼此不交的 perfect matchings。例如 $K_{2,2}$ 的兩組可取 {a1,b2} 與 {a2,b1}。若拿各匹配當一種 edge color，每個端點 incident edges 全異色，恰需 r 色。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.11 正確性與複雜度檢查

解題可用不同的最優性憑證核對結果。最短路徑用達到 d 的路徑與每邊三角不等式；MST 用保持 forest 的安全 cut；最大流用同值最小割；最大二分匹配用同大小 vertex cover，或沒有增廣路徑。

比較速度前先定義 graph representation、priority queue、input distribution 與 worst/average case。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

問題目標不能互換

MST 最小化所選生成樹全部邊的總重，不最小化某個 source 到各點的樹路徑。三角形 AB:2、BC:2、AC:3 的 MST 為 AB、BC，總重 4；從 A 到 C 的 MST 路徑長 4，卻比直接 AC 的最短距離 3 大。

最大 cardinality matching 只數配對個數；若 a1 成本 100、a2 成本 1、b1 成本 1、b2 成本 100，兩個 perfect matchings 都大小 2，成本卻為 200 與 2。只有最大流大小仍無法分辨 weighted assignment 的目標，須另加成本最小化方法。對應練習與完整詳解分別見 本章練習題 PDF、本章詳解 PDF。

參考資料與署名

Lin Cing，《離散數學》手寫筆記。 Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 13.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study